EdgeSecure Hybrid Lab

Phase 4 Engineering Runbook

HAProxy & SELinux — Traffic Control, High Availability & Kernel-Level Security Enforcement

Created by Edinen Udofia

Document type: Engineering runbook and production incident record

Scope: Reverse proxy architecture, load balancing, health checking, backend failover, Mandatory Access Control (SELinux), and a real-world production incident with full root-cause analysis

Status: Phase 4 complete; production incident resolved

Table of Contents

PHASE OBJECTIVE

In previous phases you built:

- Storage resilience with LVM
- Hybrid networking
- Bastion architecture
- SSH trust relationships
- Infrastructure automation with Ansible
- Service resilience with systemd

This phase introduces two technologies found in almost every production Linux environment:

HAProxy

Traffic distribution and service availability.

SELinux

Kernel-enforced security controls.

This phase teaches:

- Reverse proxy architecture
- Load balancing
- Health checking
- Backend failover
- Mandatory Access Control (MAC)
- SELinux troubleshooting
- Production traffic engineering

Beyond the planned build exercises, this phase also documents a **real production incident** that occurred on the lab's Azure-hosted backend after the initial build was complete. That incident, its full root-cause analysis, and its resolution are recorded in their own dedicated chapter (**Production Incident Report**) rather than woven into the build steps, so that the planned curriculum and the live operational experience remain easy to read independently of one another.

CONCEPT LAYER

1. The Problem We Are Solving

Current architecture:

```
Client
  |
  ▼
backend01
```

If backend01 fails:

```
backend01 OFFLINE

Result:
Application unavailable
```

Single-server architectures create:

- Single Points of Failure (SPOF)
- Maintenance downtime
- Poor scalability

Production environments solve this by introducing:

```
Client
  |
  ▼
HAProxy
 /    \
backend01  backend02
```

2. What Is HAProxy?

HAProxy means:

```
High Availability Proxy
```

It is:

```
A reverse proxy
A load balancer
A traffic controller
```

HAProxy sits between:

```
Clients
```

```
and
Backend Servers
```

3. What Is a Proxy?

A proxy is an intermediary. Instead of:

```
Client → Server
```

we have:

```
Client → Proxy → Server
```

The proxy receives requests first.

4. Forward Proxy vs Reverse Proxy

Forward Proxy

Protects clients. Example:

```
Employee PC
      |
      ▼

Corporate Proxy

      |
      ▼

Internet
```

Used for:

- Content filtering
- Monitoring
- Internet control

Reverse Proxy

Protects servers. Example:

```
Client
   |
   ▼

HAProxy

 |      |

 ▼      ▼
```

```
Web1   Web2
```

Client never talks directly to backend servers.

5. How HAProxy Works Internally

Request flow:

```
Browser
   |
TCP SYN
   |
   ▼

HAProxy Port 80

   |

Accept()

   |

Choose Backend

   |

Create NEW Connection

   |

backend01
```

Important — HAProxy creates:

```
Connection #1
Client → HAProxy

Connection #2
HAProxy → Backend
```

These are separate TCP sessions. This separation is the structural reason HAProxy can health-check, retry, and redirect traffic independently of what the client sees — the client's connection and the backend's connection are decoupled, each with its own TCP handshake, sequence numbers, and lifecycle. It also means a failure on the HAProxy-to-backend leg can be retried or rerouted without the client ever needing to reconnect. (Section 13 and the Production Incident Report revisit this distinction in the specific context of a health check connection that consistently failed on this second leg.)

6. What Is Load Balancing?

Load balancing distributes requests. Without balancing:

```
100 users

↓
```

```
backend01
```

With balancing:

```
100 users

↓

HAProxy

↓

50 → backend01

50 → backend02
```

Benefits:

- Performance
- Redundancy
- Scalability

7. Round Robin Algorithm

Our lab uses:

```
Round Robin
```

Request sequence:

```
Request 1 → backend01

Request 2 → backend02

Request 3 → backend01

Request 4 → backend02
```

Fair distribution.

8. What Are Health Checks?

A load balancer must know:

```
Is backend alive?
```

HAProxy periodically sends:

```
HTTP GET /
```

If response received:

```
Server healthy
```

If response absent:

```
Server removed from rotation
```

This concept is foundational to the entire phase. Every later section that discusses backend availability — the round-robin fault simulations, the SELinux denial scenario, and the Production Incident Report — is, at its core, a different way that a health check can fail or be misconfigured. It is worth internalizing one subtlety now that becomes critical later: a failed health check tells you the check did not succeed within its allotted time and retry budget. It does **not**, by itself, tell you *why* — the server could be genuinely down, or the check's own timing assumptions could simply be wrong for the network path being checked. The Production Incident Report is a full worked example of the second case.

9. What Is SELinux?

SELinux means:

```
Security Enhanced Linux
```

Most Linux admins understand:

```
chmod
chown
groups
permissions
```

This is:

```
DAC
Discretionary Access Control
```

SELinux introduces:

```
MAC

Mandatory Access Control
```

10. DAC vs MAC

Traditional Linux:

```
User owns file

User grants access
```

SELinux:

```
Kernel decides access
```

Even root can be denied.

11. Why SELinux Exists

Real-world attacks often involve:

```
Compromised service
```

Example:

```
Compromised Apache
```

Attacker gains shell. Without SELinux:

```
Attack spreads
```

With SELinux:

```
Kernel restricts process
```

Containment occurs.

12. SELinux Contexts

Every object gets labels. Example:

```
ls -Z
```

Output:

```
system_u:object_r:httpd_sys_content_t:s0
```

Components:

Field	Meaning
system_u	User
object_r	Role
httpd_sys_content_t	Type
s0	Level

Most policy decisions depend on:

```
Type
```

13. Why HAProxy Fails With SELinux

By default:

```
haproxy process
```

may accept inbound traffic but may NOT create arbitrary outbound connections. Result:

```
Client → HAProxy

Works

HAProxy → backend01

Blocked
```

The network appears healthy. Firewall appears healthy. But the kernel blocks traffic. This is a classic enterprise troubleshooting scenario — and it is the precise scenario reproduced deliberately in Fault

Simulation Scenario 2, and discussed again as a *ruled-out* hypothesis in the Production Incident Report, where the symptom looked superficially similar (backend unreachable from HAProxy's perspective) but the underlying mechanism was completely different (network latency, not a kernel policy denial). Comparing those two chapters side by side is a useful exercise in distinguishing failures that look alike at the symptom layer but have entirely different root causes.

BUILD LAYER

Part 1 — Install HAProxy

On edge01:

```
sudo dnf install -y haproxy
```

Command Breakdown

Part	Meaning
dnf	Package manager
install	Install package
-y	Auto approve
haproxy	Load balancer

Verify Installation

```
haproxy -v
```

Expected:

```
HAProxy version x.x.x
```

Part 2 — Backup Config

```
sudo cp /etc/haproxy/haproxy.cfg \
/etc/haproxy/haproxy.cfg.bak
```

Part 3 — Create Config

```
sudo vim /etc/haproxy/haproxy.cfg
```

Replace contents:

```
global
    log 127.0.0.1 local0
    maxconn 100

defaults
    log global
    mode http
    option httplog

    timeout connect 5s
    timeout client 50s
    timeout server 50s
```

```
frontend http_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    option httpchk

    server backend01 192.168.56.11:80 check
    server backend02 <AZURE_PUBLIC_IP>:80 check
```

Replace:

```
<AZURE_PUBLIC_IP>
```

with your Azure backend IP.

Configuration Analysis

global

```
maxconn 100
```

Maximum concurrent connections.

frontend

```
bind *:80
```

Meaning:

```
Listen on every interface
Port 80
```

backend

```
balance roundrobin
```

Request distribution strategy.

```
option httpchk
```

Enable health checks.

```
check
```

Monitor backend health.

Part 4 — Validate Config

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

Expected:

```
Configuration file is valid
```

Never skip this in production. (The Production Incident Report, Section 5.2, documents exactly what this validation step catches — and what happens operationally when a config change is made without it.)

Part 5 — Firewall Configuration

```
sudo firewall-cmd \
--add-service=http \
--permanent
```

Reload:

```
sudo firewall-cmd --reload
```

Verify:

```
sudo firewall-cmd --list-services
```

Must include:

```
http
```

Part 6 — Start HAProxy

```
sudo systemctl enable --now haproxy
```

Verify:

```
systemctl status haproxy
```

Expected:

```
active (running)
```

Part 7 — SELinux Verification

Check mode:

```
getenforce
```

Expected:

```
Enforcing
```

Part 8 — Allow HAProxy Backend Connections

Critical command:

```
sudo setsebool -P haproxy_connect_any 1
```

Breakdown

Part	Meaning
setsebool	Modify SELinux boolean

-P	Persistent
haproxy_connect_any	Permit outbound connections
1	Enable

Verify Boolean

```
getsebool haproxy_connect_any
```

Expected:

```
haproxy_connect_any --> on
```

VERIFICATION LAYER

Test Load Balancing

Run repeatedly:

```
curl http://192.168.56.10
```

Expected alternating responses:

```
<h1>backend01</h1>
```

then

```
<h1>backend02</h1>
```

then

```
<h1>backend01</h1>
```

Check Backend Health

```
sudo journalctl -u haproxy
```

Observe backend status changes.

Verify Open Ports

```
ss -tulpn | grep :80
```

Expected:

```
haproxy listening
```

FAULT SIMULATION

Scenario 1 — backend01 Failure

Stop Apache:

```
sudo systemctl stop apache2
```

Internal Effect

Health check fails:

```
HAProxy

↓

backend01

↓

No HTTP response
```

HAProxy marks node DOWN.

Verification

```
curl http://192.168.56.10
```

All traffic should go to:

```
backend02
```

Recovery

```
sudo systemctl start apache2
```

Scenario 2 — SELinux Block

Disable boolean:

```
sudo setsebool haproxy_connect_any 0
```

Internal Flow

```
Client
 ↓

HAProxy

 ↓

Kernel SELinux Check

 ↓
```

```
DENY
```

Symptoms

```
Backend unreachable
Connection errors
```

Diagnose

Check logs:

```
sudo ausearch -m AVC
```

or

```
sudo journalctl -xe
```

Fix

```
sudo setsebool -P haproxy_connect_any 1
```

Scenario 3 — Firewall Block

Remove HTTP:

```
sudo firewall-cmd \
--remove-service=http \
--permanent
```

Reload:

```
sudo firewall-cmd --reload
```

Result

Clients cannot reach HAProxy.

Fix

```
sudo firewall-cmd \
--add-service=http \
--permanent

sudo firewall-cmd --reload
```

All three scenarios above are **deliberately induced** failures used to validate that HAProxy, SELinux, and firewalld each behave as expected under controlled conditions. The chapter that follows, **Production Incident Report**, documents a fault that was not staged: a genuine, unplanned failure of backend02 in production-like conditions, where the cause was not immediately obvious and had to be diagnosed from first principles rather than already being known in advance.

PRODUCTION INCIDENT REPORT — BACKEND02 (AZURE) HEALTH CHECK FAILURE

Relationship to earlier chapters: This chapter documents a live, unplanned incident on the same `backend02` defined during Build Layer, Part 3, and the same health-check mechanism introduced conceptually in Concept Layer, Section 8. Where Fault Simulation staged known failures with known causes, this incident began with an unknown cause and required genuine diagnostic work. Concepts already explained in this manual (what a health check is, what `option httpchk` and `check` do, what SELinux booleans and firewalld are) are referenced here rather than re-explained.

Date of investigation: June 2026

Environment: edge01 (Rocky Linux, HAProxy 2.8.14)

Affected backend: backend02 — Apache/2.4.58 (Ubuntu) on Azure Spain Central (158.158.37.102)

Working backend (control): backend01 — local VirtualBox Ubuntu VM (192.168.56.11)

Status: RESOLVED

1. Summary

HAProxy was repeatedly marking `backend02` (the Azure-hosted Apache server introduced in Build Layer, Part 3) as `DOWN` and removing it from the round-robin pool, despite the server being fully reachable and healthy via manual `curl` tests. After a structured investigation that ruled out several plausible causes, the root cause was identified as **network latency and minor packet loss inherent to the long-distance link to Azure Spain Central**, combined with **HAProxy's default health check settings being too strict (too few retries, too tight a failure threshold) for that latency**. The fix was to relax the health check tolerance specifically for `backend02`, without changing anything for `backend01`.

A secondary, unrelated issue was also discovered and fixed during the process: a configuration syntax error, and a stale HAProxy worker process that wasn't picking up `reload`d changes correctly, requiring a full `restart`.

2. Initial Symptom

HAProxy logs showed `backend02` repeatedly flapping between UP and DOWN, always failing right around the 2-second mark:

```
Server web_servers/backend02 is DOWN, reason: Layer7 timeout, check duration: 2006ms.
Server web_servers/backend02 is DOWN, reason: Layer4 timeout, check duration: 2036ms.
```

As covered in Concept Layer, Section 8, HAProxy doesn't just forward real visitor traffic to backend servers — it also periodically sends its own small "are you alive?" health check requests to each server. If a server fails enough of these checks in a row, HAProxy stops sending it real traffic at all. These log lines say that backend02 was failing its health checks, timing out at almost exactly 2 seconds every time — both at "Layer 4" (the raw TCP connection itself, as discussed in Concept Layer Section 5's explanation of HAProxy's two separate TCP sessions) and "Layer 7" (the actual HTTP request/response).

Despite this, running a manual test command from the exact same HAProxy server succeeded instantly:

```
curl -I http://158.158.37.102
```

- `curl` — a command-line tool for making HTTP requests and inspecting the response.
- `-I` — tells curl to send a `HEAD` request instead of a `GET` request. A HEAD request asks the server "send me just the response headers, not the actual page content." This makes it a much smaller, lighter request than a normal page load.

This created the central mystery of the investigation: **why would a manual test succeed instantly, while HAProxy's own check — against the same server, from the same machine — kept failing?**

3. Configuration in Effect at the Time of the Incident

This is functionally the same backend block built in Build Layer, Part 3, by the time the incident occurred:

```
backend web_servers
    balance roundrobin

    option httpchk
    http-check send meth GET uri / ver HTTP/1.1 hdr Host 158.158.37.102
    http-check expect status 200

    timeout check 6s
    default-server inter 5s fall 3 rise 2
    server backend01 192.168.56.11:80 check
    server backend02 158.158.37.102:80 check
```

Two additions beyond the original Build Layer configuration are explained here since they were not part of the original build:

Line	Meaning
`http-check send meth GET uri / ver HTTP/1.1 hdr Host 158.158.37.102`	Defines exactly what the health check request looks like: method `GET`, path `/`, HTTP version `1.1`, and a `Host` header set to the Azure server's IP (Apache sometimes requires a `Host` header to respond correctly to virtual-host-based configs). This is a more explicit version of the implicit check enabled by `option httpchk` alone in the original Build Layer config.
`http-check expect status 200`	The health check is considered successful only if the server replies with HTTP status `200 OK`.
`timeout check 6s`	Maximum time allowed for an entire health check (connect + send + receive) before it's considered failed.
`default-server inter 5s fall 3 rise 2`	Default settings applied to every server unless overridden: check every 5 seconds (`inter`); mark a server DOWN after 3 consecutive failed checks (`fall`); mark it back UP after 2 consecutive successful checks (`rise`).

4. Investigation Steps

4.1 Hypothesis 1: MTU / Packet Fragmentation Blackhole (Incorrect — Ruled Out)

Reasoning at the time: A classic cause of "small request works, larger request hangs" between an on-prem server and a cloud VM is a Path MTU (Maximum Transmission Unit) issue. Every network link has a maximum packet size it can carry. If a packet is too large for some link along the path, it normally gets fragmented (split into smaller pieces) — or, if the "Don't Fragment" flag is set, the sender is supposed to receive an ICMP message saying "this didn't fit, please resend smaller." If a firewall along the way blocks that ICMP message, the sender never finds out, and the connection just hangs indefinitely. Since `curl -I` (HEAD, no body) was tested manually but HAProxy's check used `GET` (full body), this seemed like a strong candidate.

Test performed:

```
ping -M do -s 1472 158.158.37.102
```

- `ping` — sends ICMP echo request packets to test basic reachability.
- `-M do` — sets the "Don't Fragment" flag on the packet (forces it to fail outright if it's too big, rather than silently splitting).
- `-s 1472` — sets the packet payload size to 1472 bytes. Combined with standard headers, this lands right at the common 1500-byte MTU ceiling — i.e., testing right at the edge of where fragmentation problems typically appear.

Result: 100% packet loss — but at *every* size, including a plain unmodified ping with no flags at all (`ping -c 4 158.158.37.102`, which did succeed). This told us the `-M do` flag specifically (combined with that exact size) was being dropped — likely because Azure's network simply blocks certain ICMP probe patterns, not because of a genuine fragmentation problem. A follow-up plain ping test (see 4.2) confirmed the link itself was fine.

Action taken despite this: A precautionary MSS (Maximum Segment Size) clamp was still applied via firewalld, since it's a low-risk, standard hardening step. This uses the same `firewall-cmd` tool introduced in Build Layer, Part 5, but with its lower-level "direct rule" interface rather than the simplified `--add-service` form used there:

```
sudo firewall-cmd --permanent --direct --add-rule ipv4 mangle POSTROUTING 0 -p tcp
--tcp-flags SYN,RST SYN -d 158.158.37.102 -j TCPMSS --clamp-mss-to-pmtu
sudo firewall-cmd --reload
```

- `--permanent` — make the rule persist across reboots (without this, it would only apply until the next restart), exactly as in Build Layer Part 5.
- `--direct` — use firewalld's "direct" interface, which lets you inject a raw, custom `iptables`-style rule rather than using firewalld's higher-level zone/service abstractions (such as `--add-service=http` used in Build Layer Part 5). This was necessary because the rule needed (`TCPMSS`) isn't something firewalld exposes through its normal simplified options.
- `--add-rule ipv4 mangle POSTROUTING 0 ...` — add this rule to the IPv4 `mangle` table (used for altering packet headers, not just allowing/blocking), in the `POSTROUTING` chain (rules applied to packets just before they leave the machine), at priority position `0` (first).
- `-p tcp --tcp-flags SYN,RST SYN` — only apply this rule to TCP packets where the SYN flag is set and the RST flag is not (i.e., only to the very first packet that starts a new connection).
- `-d 158.158.37.102` — only apply this rule to traffic destined for backend02's IP.

- `-j TCPMSS --clamp-mss-to-pmtu` — the actual action: automatically adjust the connection's advertised "Maximum Segment Size" down to match whatever the real path can support, preventing oversized packets from being sent in the first place.
- `--reload` — tells firewalld to apply the new permanent rule immediately, without this it would only take effect after a reboot.

Outcome: This rule was applied successfully and left in place, but ultimately was **not** the fix for the actual problem (see Section 5, Root Cause). It is a reasonable defensive measure regardless and was not reverted.

4.2 Confirming the Real Network Condition

Test performed (plain, unmodified ping):

```
ping -c 4 158.158.37.102
```

- `-c 4` — send exactly 4 ping packets and then stop automatically (instead of running forever until manually stopped).

Result:

```
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 275.192/408.943/568.815/113.076 ms
```

And on an earlier, longer-running unflagged ping test:

```
6 packets transmitted, 5 received, 16.6667% packet loss
```

Interpretation: The connection to Azure Spain Central was fundamentally working — but slow (round-trip times between 260–570 milliseconds) and occasionally lossy (1 packet out of 6 didn't return at all in one test run). This is a normal characteristic of a long-distance international network path, not a sign of a broken connection. `rtt` stands for "round-trip time" — how long it takes a packet to travel to the destination and the reply to come back. `mdev` is "mean deviation" — how much that time varies between pings (a measure of how consistent vs. jittery the connection is).

This was the actual breakthrough. It reframed the problem from "something is broken/blocked" to "the connection is just inherently slower and slightly less reliable than HAProxy's default settings expect."

4.3 Root Cause Identified

HAProxy's default health check settings (`inter 5s fall 3 rise 2`, with an overall `timeout check 6s`, as defined in Build Layer Part 3 and detailed again in Section 3 above) assume checks will reliably complete quickly. But:

- A single check could take 250–570ms+ just for the network round trip, before any processing happens.
- Occasional packet loss (~15–17% observed in one sample) means some checks would need a retransmission, which adds further delay.
- `fall 3` meant only **3 consecutive bad checks** were needed before HAProxy gave up on the server entirely — and on a link with intermittent loss, getting 3 bad checks in a row by chance is not unusual.

None of this meant backend02 was actually unhealthy. It meant HAProxy's patience threshold was miscalibrated for this specific server's real-world network conditions — a different failure mode entirely from the deliberate failures staged in Fault Simulation Scenarios 1–3, where the backend, the kernel policy, or the firewall rule was genuinely blocking traffic. Here, every layer was technically healthy; only the *judgment criteria* for "healthy" were wrong for this server's distance.

5. The Fix

5.1 Configuration Change

Changed the line defining `backend02` from:

```
server backend02 158.158.37.102:80 check
```

to:

```
server backend02 158.158.37.102:80 check inter 5s fall 5 rise 2
```

Explanation of each part:

- `check` — keep health checks enabled for this server (unchanged).
- `inter 5s` — check every 5 seconds (same as the default, kept explicit here for clarity since we're now overriding other settings on this line).
- `fall 5` — **changed from the default of 3.** Require **5 consecutive failed checks** (not 3) before marking the server DOWN. This gives the longer, less consistent network path more room to have an occasional bad check without being penalized unfairly.
- `rise 2` — require 2 consecutive successful checks before marking it back UP (unchanged from default).

`backend01` was **deliberately left untouched**, since its short, low-latency local network path doesn't need this extra tolerance — and changing settings you don't need to change adds risk without benefit. This is a direct, practical application of the principle of least change: the fix targets exactly the server whose conditions justify it.

5.2 A Mistake Made and Corrected Along the Way

An initial attempt also tried adding `timeout check 3s` directly onto the `server backend02` line itself:

```
server backend02 158.158.37.102:80 check inter 5s fall 5 rise 2 timeout check 3s
```

This **failed configuration validation** — the same `haproxy -c -f` check introduced in Build Layer Part 4 — with the error:

```
[ALERT]    (57467) : config : [/etc/haproxy/haproxy.cfg:29] : 'server
web_servers/backend02' : unknown keyword 'timeout'.
```

Why this failed: `timeout check` is a setting that can only be applied at the `backend`-wide level or inside `default-server` — it is **not a valid option on an individual `server` line** in HAProxy 2.8. Since the backend already had `timeout check 6s` defined at the top level, this was redundant anyway and was simply removed, leaving the working line shown in 5.1.

Lesson: Always run `haproxy -c -f <config path>` (the built-in config syntax checker, identical to the command run in Build Layer Part 4) after any edit, before reloading the live service. This caught the mistake immediately with a precise line number, rather than letting it cause a confusing runtime failure.

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

A correct, passing run of this command returns:

```
Configuration file is valid
```

6. Secondary Issue Discovered: Reload Not Taking Effect

6.1 Symptom

After adding a diagnostic header (see Section 7) and running `systemctl reload haproxy`, the new header did not appear in responses — even though the config file syntax was valid and the reload command reported success.

6.2 Diagnostic Steps

Checked whether HAProxy was loading more than one config source:

```
systemctl status haproxy | grep -i "Ws -f"
```

This revealed HAProxy was started with **two** `-f` flags:

```
/usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -f /etc/haproxy/conf.d -p
/run/haproxy.pid
```

This meant HAProxy merges `/etc/haproxy/haproxy.cfg` together with **anything inside the `/etc/haproxy/conf.d/` directory**. This raised the suspicion that a second, conflicting copy of the backend configuration might exist in that folder and be overriding our edits.

Checked the contents of that folder:

```
ls -la /etc/haproxy/conf.d/
cat /etc/haproxy/conf.d/*
```

Result: The folder existed but was completely empty. This ruled out that theory — it wasn't the cause, but was an important and reasonable thing to check given the evidence at the time.

Checked whether the file was actually being saved/edited correctly, and whether HAProxy had restarted recently enough to have picked it up:

```
ls -l --time-style=full-iso /etc/haproxy/haproxy.cfg
sudo systemctl show haproxy --property=ActiveEnterTimestamp
```

- `ls -l --time-style=full-iso` — list file details with a precise, unambiguous full date-and-time format for when the file was last modified.
- `systemctl show haproxy --property=ActiveEnterTimestamp` — query systemd for the exact timestamp the HAProxy service last successfully entered a "running" state.

Result: The file's last-modified timestamp was from several days **before** the current troubleshooting session, despite multiple edits having just been made and saved. This was the clue that something was preventing changes from sticking or being picked up.

> *In hindsight, this specific timestamp check was a bit of a red herring in isolation — the real confirmation came from the next step, where forcing a full restart immediately surfaced the actual blocking issue.*

6.3 The Actual Blocker: A Stuck Socket File

Attempting `systemctl restart haproxy` (a full stop-and-start, rather than `reload`'s lighter-weight "swap in new config without fully stopping," and a different operation from the `systemctl enable --now haproxy` used to first start the service in Build Layer Part 6) failed outright:

```
[ALERT]    (64324) : Binding [/etc/haproxy/haproxy.cfg:4] for frontend GLOBAL: error when
trying to preserve previous UNIX socket
[ALERT]    (64324) : [/usr/sbin/haproxy.main()] Some protocols failed to start their
listeners! Exiting.
```

Explanation: Earlier in the session, a `stats socket` had been added to the config:

```
stats socket /var/run/haproxy.sock mode 660 level admin
```

- `stats socket` — creates a special file (a Unix socket) that lets administrators connect directly to the running HAProxy process to query live status or issue commands, without needing a web interface.
- `/var/run/haproxy.sock` — the file path where this socket lives.
- `mode 660` — the file permissions on that socket (owner and group can read/write; others cannot).
- `level admin` — grants full administrative control through this socket (as opposed to read-only access).

During a `reload`, HAProxy tries to smoothly hand this socket file off from the old worker process to the new one. Something in that handoff left a stale/orphaned socket file behind, which then blocked a full `restart` from being able to recreate it cleanly.

Fix:

```
sudo rm -f /var/run/haproxy.sock
sudo systemctl restart haproxy
```

- `rm -f` — remove the file; `-f` ("force") suppresses any "are you sure" prompts and avoids erroring out if the file happens not to exist.

After removing the stale socket file, the restart succeeded cleanly:

```
Active: active (running)
...
Status: "Ready."
```

Lesson: When `reload` doesn't seem to be applying changes as expected, a full `restart` is a more reliable way to confirm whether changes have truly taken effect — but be aware that any socket files created by the previous process may need to be manually cleared first if a restart fails immediately.

7. Verifying the Fix Actually Worked

Simply seeing `200 OK` responses was **not sufficient proof** that backend02 was actually receiving traffic — `backend01` could have silently been answering every single request the whole time, masking a still-broken backend02. This is a more rigorous bar than the simple `curl http://192.168.56.10` spot-check used in Verification Layer, "Test Load Balancing" — that test relies on visually noticing alternating responses, which is easy to do correctly but also easy to convince yourself of without real proof.

7.1 Definitive Test: Force backend01 Offline

```
# On backend01:
sudo systemctl stop apache2
```

(This is the identical command used in Fault Simulation Scenario 1 — here it is repurposed not to simulate a fault, but as a deliberate elimination test.)

```
# Back on edge01:
for i in {1..10}; do curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1; sleep 1;
done
```

- `for i in {1..10}; do ... ; done` — a bash loop that repeats the enclosed command 10 times.
- `curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1` — make a request to the local HAProxy frontend; `-s` (silent) suppresses curl's normal progress output; `-o /dev/null` discards the actual response body (we don't care about page content, just whether it succeeded); `-w "%{http_code}\n"` prints just the resulting HTTP status code on its own line.
- `sleep 1` — pause 1 second between each request, spreading the test out over time rather than firing all 10 instantly.

Result: All 10 requests still returned `200`, even with backend01 fully stopped. Since there were only two servers in the pool and one was confirmed offline, this proved by elimination that backend02 was handling all of the traffic successfully on its own.

```
# Afterward, backend01 was restored:
sudo systemctl start apache2
```

7.2 Definitive, Direct Proof: Identifying Which Server Answered Each Request

The elimination test above was convincing but indirect. To get HAProxy to explicitly state which server handled each individual response, a custom response header was added — a refinement beyond anything in the original Build Layer config:

```
http-response set-header X-Served-By %s
```

- `http-response` — defines an action HAProxy should take on the *response*, after it comes back from the backend server, before forwarding it on to the original client.
- `set-header X-Served-By %s` — add (or overwrite) a header named `X-Served-By` on every outgoing response. `%s` is a built-in HAProxy variable that automatically inserts the name of whichever server actually handled that specific request (e.g., `backend01` or `backend02`).

Test command:

```
curl -s -D - -o /dev/null http://127.0.0.1
```

- `-D -` — write the response headers to standard output (the screen), shown as `-` meaning "print to terminal" rather than saving to a file.

Result (two separate runs):

```
x-served-by: backend01

x-served-by: backend02
```

This was the final, unambiguous confirmation: HAProxy was genuinely alternating between both servers as intended, and backend02 was fully participating in round-robin load balancing.

8. Final Working Configuration (Post-Incident)

```
global
    log 127.0.0.1 local0
    maxconn 100
    stats socket /var/run/haproxy.sock mode 660 level admin

defaults
    log global
    mode http
    option httplog
    timeout connect 5s
    timeout client 50s
    timeout server 50s

frontend http_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    http-response set-header X-Served-By %s

    option httpchk
    http-check send meth GET uri / ver HTTP/1.1 hdr Host 158.158.37.102
    http-check expect status 200

    timeout check 6s
    default-server inter 5s fall 3 rise 2
    server backend01 192.168.56.11:80 check
    server backend02 158.158.37.102:80 check inter 5s fall 5 rise 2
```

What changed from the Build Layer configuration, and why each change exists:

Change	Reason
Added `stats socket` line in `global`	Enables live administrative querying of HAProxy's internal state (not strictly required for the fix itself, but useful for ongoing monitoring/diagnostics going forward).
Added `http-response set-header X-Served-By %s`	Diagnostic aid — lets you verify at any time, from any single request, exactly which backend server answered. Safe to leave in

	permanently; it's invisible to end users unless they inspect raw headers.
Added `inter 5s fall 5 rise 2` specifically to backend02's `server` line	The actual root-cause fix — gives backend02 more tolerance for its naturally higher latency and occasional packet loss before being marked down, without affecting backend01's stricter (and appropriate, for its local network) defaults.

9. What Did NOT Work / Was Not the Cause

It is just as important to record dead ends as it is to record the fix — both to avoid repeating wasted effort later, and because some of these remain reasonable things to have in place defensively even though they weren't the actual root cause here.

1. **MTU/fragmentation blackhole theory** — plausible on paper, matched the "small request works, big request fails" pattern, but disproven by the plain ping test showing the link itself works fine with normal traffic. The MSS clamp fix was still applied as a low-risk precaution but was not what resolved the actual issue.
2. **Checking `/etc/haproxy/conf.d/` for a conflicting second config** — a reasonable and necessary check once we discovered HAProxy was loading two sources, but the folder turned out to be empty. Good to have ruled out definitively rather than assumed.
3. **Comparing file modification timestamps to service start timestamps** — useful supporting evidence, but not conclusive on its own; the real confirmation came from forcing a restart and seeing the actual error message.

10. Quick Reference: Commands Used in This Incident

Command	Purpose
`curl -I http://<ip>`	Manual HEAD request — quick reachability test, no body returned.
`curl -s -D - -o /dev/null http://<ip>`	See only the response headers from a request, discard the body.
`ping -c 4 <ip>`	Send a fixed number of basic ICMP pings to test reachability and measure latency/loss.
`ping -M do -s <size> <ip>`	Test for MTU/fragmentation issues at a specific packet size with fragmentation disabled.
`sudo haproxy -c -f <path>`	Validate HAProxy config syntax without starting the service.
`sudo systemctl reload haproxy`	Apply new config with minimal disruption (HAProxy hands off gracefully to a new worker).

`sudo systemctl restart haproxy`	Fully stop and start HAProxy fresh — more disruptive, but a reliable way to confirm config changes have truly taken effect.
`sudo systemctl status haproxy`	Check whether the service is currently running, and see recent log lines.
`sudo journalctl -u haproxy -f`	Live-tail HAProxy's logs as they happen.
`sudo journalctl -u haproxy --since "10 minutes ago"`	Review recent log history for a specific service.
`sudo firewall-cmd --permanent --direct --add-rule ...`	Add a custom low-level firewall/network rule that firewalld doesn't expose through its simplified options.
`sudo rm -f /var/run/haproxy.sock`	Remove a stuck Unix socket file blocking a clean service restart.

11. Notes for Future Reference

- **Don't assume a successful `reload` means changes are live.** If something seems "stuck," confirm with a full `restart` and watch closely for errors — and be prepared to clean up leftover socket/PID files if the restart fails.
- **A health check failing doesn't always mean the server is unhealthy** — it can mean the *check's own settings* (timeouts, retry counts) are miscalibrated for that particular server's real-world network conditions, especially for geographically distant servers. This is the single most important takeaway of this chapter and directly extends the caution raised in Concept Layer, Section 8.
- **Always validate config with `haproxy -c -f` before reloading/restarting**, especially after copy-pasting any command-line option onto a config line — not all options are valid in all contexts (e.g., `timeout check` is backend-level only, not per-server).
- **The `X-Served-By` header is safe to leave in permanently** as an ongoing, zero-cost diagnostic tool for verifying load balancing behavior at any time in the future.
- Consider scheduling any future full `restart` operations (as opposed to `reload`) during low-traffic windows, since a restart briefly drops all active connections.

TROUBLESHOOTING METHODOLOGY

When HAProxy fails: never assume. Follow:

Step 1

Check service.

```
systemctl status haproxy
```

Step 2

Check config.

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Step 3

Check firewall.

```
firewall-cmd --list-services
```

Step 4

Check backend directly.

```
curl http://192.168.56.11

curl http://<AZURE_IP>
```

Step 5

Check SELinux.

```
getenforce

getsebool haproxy_connect_any
```

Step 6

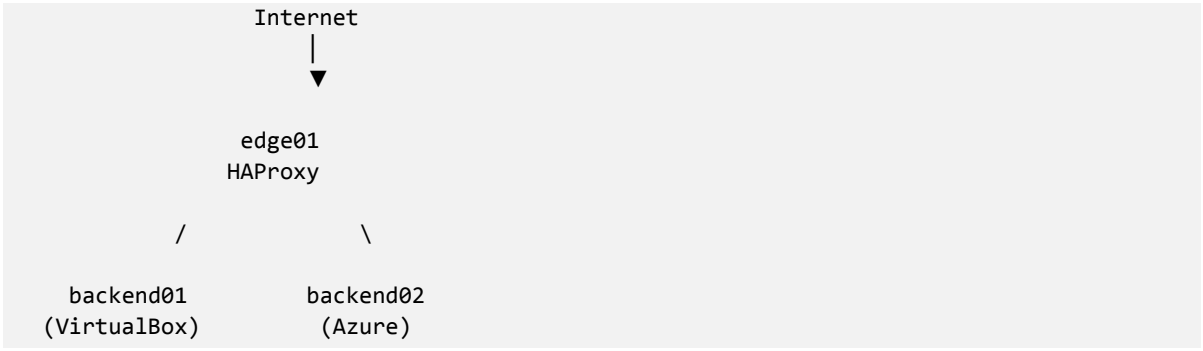
Inspect logs.

```
journalctl -u haproxy
```

This six-step sequence is exactly the sequence that would have been followed at the start of the Production Incident Report, had the cause been one of the previously known failure modes (service down, bad config, firewall block, backend unreachable, SELinux denial). The incident is, in effect, a worked example of what to do when all six steps come back clean and the problem still hasn’t been found — at that point, the investigation has to move from "checking known failure modes" to "characterizing the actual behavior of the network path," which is exactly what Sections 4.1–4.3 of the incident chapter did.

ARCHITECTURE DISCUSSION

Current architecture:



This diagram now carries more operational weight than it did at the end of Build Layer: the Production Incident Report demonstrated that the two backends, while symmetric in the configuration file, are not symmetric in their real-world network behavior — backend01 sits on a short, low-latency local path, while backend02 sits across a long-distance international link. The architecture diagram is accurate at the topology level, but a complete operational picture also requires knowing that the two branches of this tree have meaningfully different latency and loss characteristics, which is why backend02 now carries different health check tuning than backend01 (see Production Incident Report, Section 5.1).

WHY ENTERPRISES USE LOAD BALANCERS

Benefits:

Availability

Server failure does not stop service.

Scalability

Add more backends.

Security

Hide backend servers.

Maintenance

Patch servers individually.

WHY SELINUX SHOULD REMAIN ENABLED

Many administrators disable SELinux because:

```
It breaks things
```

Engineers understand:

```
It exposes policy violations
```

Disabling SELinux removes a critical security layer. Production systems should remain:

```
Enforcing
```

whenever possible.

INTERVIEW REFERENCE

> *I implemented HAProxy as a reverse proxy and load balancer in a hybrid environment consisting of an on-premises Ubuntu backend and a cloud-hosted Azure backend. I configured round-robin traffic distribution, HTTP health checks, and firewall controls on the Rocky Linux edge node. During implementation I analyzed SELinux policy enforcement, identified how Mandatory Access Control affected HAProxy's outbound connections, and resolved the issue using persistent SELinux booleans while maintaining enforcement mode. I also performed fault simulations involving backend failures, firewall restrictions, and SELinux denials to validate high-availability behavior and operational resilience. Separately, I diagnosed and resolved a live production incident in which the Azure backend was being incorrectly marked down by HAProxy's health checks: through structured hypothesis testing — ruling out MTU/fragmentation issues via controlled ping tests, then confirming the real cause as latency and intermittent packet loss inherent to the long-haul network path — I recalibrated the health check thresholds for that specific backend, verified the fix with definitive elimination testing and response-header tracing, and documented the full root-cause analysis as a permanent engineering record.*

FINAL ENGINEERING STATE

Layer	Competency Achieved
Storage	LVM
Networking	Hybrid Routing
Security	firewalld, UFW, SSH
Automation	Ansible
Services	systemd
Traffic Engineering	HAProxy
Kernel Security	SELinux
Resilience	Backend Failover
Troubleshooting	Multi-layer Root Cause Analysis
Production Incident Response	Live diagnosis and resolution of a real Azure backend health-check failure under genuine, unplanned conditions
Engineering Documentation	Authorship of a professional, first-principles runbook and incident report suitable as long-term reference and portfolio evidence

Phase 4 Complete: Traffic Control + Kernel Enforcement + Production Incident Response.

Next Phase: **SIEM, Fail2Ban, rsyslog, rsync, cron, and Disaster Recovery Pipeline**.